1. Represent the input

In [368…
```
sentence = [("The", "DT"), ("dog", "NN"), ("chased", "VBD"),
            ("the", "DT"), ("cat", "NN")]
sentence1 = [("The", "DT"), ("dog", "NN"), ("sees", "VBZ"),
             ("the", "DT"), ("cat", "NN")]
sentence2 = [("The", "DT"), ("big", "JJ"), ("dog", "NN"),
             ("sees", "VBZ"), ("a", "DT"), ("small", "JJ"),
             ("cat", "NN")]
sentence3 = [("The", "DT"), ("cat", "NN"), ("sleeps", "VBZ"),
             ("on", "IN"), ("the", "DT"), ("bed", "NN")]
sentence4 = [("The", "DT"), ("dog", "NN"), ("sees", "VBZ"),
             ("a", "DT"), ("cat", "NN"), ("and", "CC"),
             ("a", "DT"), ("bird","NN")]
sentence5 = [("The", "DT"), ("boy", "NN"), ("sees", "VBZ"),
             ("the", "DT"), ("dog", "NN"), ("running", "VBG")]
```

In [369…
```
# debug print
def print_parser(stack, buffer, relations):
    print(f"stack: {stack}")
    print(f"buffer: {buffer}")
    print(f"relations: {relations}")
    print()
```

1. Implement the transitions

In [370…
```
def shift(stack, buffer):
    token =[]
    token = buffer[0]
    stack = stack + [token]
    buffer = buffer[1:]
    return stack, buffer

def left_arc(stack, relations):
    transition = []
    dependent = stack[-1]
    head = stack[-2]
    stack.pop()
    arc = " --> "
    transition = [head[0] + arc + dependent[0]]
    relations = relations + transition
    return (stack, relations)

def right_arc(stack, relations):
    transition = []
    dependent = stack[-2]
    head = stack[-1]
    stack.pop(-2)
    arc = " --> "
    transition = [head[0] + arc + dependent[0]]
    relations = relations + transition
    return (stack, relations)
```

1. Implement the parser

In [371…
```
def parser(sentence):
    # Gruppen von POS-Tags
    nouns = ["NN", "NNS", "NNP", "NNPS", "PRP"]
    verbs = ["VB", "VBD", "VBZ", "VBP", "VBN", "VBG"]
```

```
adjectives = ["JJ", "JJR", "JJS"]
determiners = ["DT", "PRP$"]
prepositions = ["IN"]
conjunctions = ["CC"]

'''
Parse Regeln:
nouns --> determiner
nouns --> adjectives
nouns --> prepositions
nouns --> conjunctions
verbs --> nouns
1.nomen --> 2.nomen, wenn es sonst nirgendwo hingehört
ROOT --> verb, wenn buffer leer ist
'''


# 1. stack, buffer und relations initialisieren
stack = [("ROOT", "ROOT")]
buffer = sentence
relations = []

# 2. Transitionen anwenden, bis der buffer leer ist
#    und nur noch ROOT auf dem stack liegt
counter = 0
while len(buffer) > 0 or len(stack) > 1:

    # nur ROOT auf dem stack: nächstes Wort holen
    if len(stack) == 1:
        transition = "SHIFT"
        stack, buffer = shift(stack, buffer)

    # Determinierer --> Nomen
    elif stack[-2][1] in determiners and stack[-1][1] in nouns:
        transition = "RIGHT-ARC"
        stack, relations = right_arc(stack, relations)

    # Adjektiv --> Nomen
    elif stack[-2][1] in adjectives and stack[-1][1] in nouns:
        transition = "RIGHT-ARC"
        stack, relations = right_arc(stack, relations)

    # Präposition --> Nomen
    elif stack[-2][1] in prepositions and stack[-1][1] in nouns:
        transition = "RIGHT-ARC"
        stack, relations = right_arc(stack, relations)

    # Konjunktion --> zum zweiten Nomen
    elif stack[-2][1] in conjunctions and stack[-1][1] in nouns:
        transition = "RIGHT-ARC"
        stack, relations = right_arc(stack, relations)

    # Nomen vor Verb --> ist das Subjekt
    elif stack[-2][1] in nouns and stack[-1][1] in verbs:
        transition = "RIGHT-ARC"
        stack, relations = right_arc(stack, relations)

    # Nomen nach Nomen --> gehört zum ersten Nomen
    # (nur wenn der buffer leer ist, damit das zweite Nomen vollständig ist)
    elif stack[-2][1] in nouns and stack[-1][1] in nouns and len(buffer) == 0:
        transition = "LEFT-ARC"
        stack, relations = left_arc(stack, relations)

    # Nomen nach Verb --> ist das Objekt
    # (nur wenn der buffer leer ist, damit das Objekt vollständig ist)
    elif stack[-2][1] in verbs and stack[-1][1] in nouns and len(buffer) == 0:
```

```
            transition = "LEFT-ARC"
            stack, relations = left_arc(stack, relations)

        # Verb ist der head des Satzes
        elif stack[-2][1] == "ROOT" and stack[-1][1] in verbs and len(buffer) == 0:
            transition = "LEFT-ARC"
            stack, relations = left_arc(stack, relations)

        # Wenn keine Regel passt, aber buffer hat noch Wörter, nächstes Wort holen
        elif len(buffer) > 0:
            transition = "SHIFT"
            stack, buffer = shift(stack, buffer)

        # Wenn keine Regel passt und der buffer ist leer, wird das zweite element head.
        else:
            transition = "LEFT-ARC"
            stack, relations = left_arc(stack, relations)
        counter += 1
        print(f"{counter}. {transition}")
        # print_parser(stack, buffer, relations)   # debug print

    # 3. Return the resulting dependency arcs.
    return relations
```

In [372…

```
in_sentence = sentence5
print(f"Sentence: {in_sentence}")
relations = parser(in_sentence)
print()
print("dependency arcs:")
for relation in relations:
    print(relation)
```

```
Sentence: [('The', 'DT'), ('boy', 'NN'), ('sees', 'VBZ'), ('the', 'DT'), ('dog', 'NN'), ('runn
ing', 'VBG')]
1. SHIFT
2. SHIFT
3. RIGHT-ARC
4. SHIFT
5. RIGHT-ARC
6. SHIFT
7. SHIFT
8. RIGHT-ARC
9. SHIFT
10. RIGHT-ARC
11. LEFT-ARC
12. LEFT-ARC

dependency arcs:
boy --> The
sees --> boy
dog --> the
running --> dog
sees --> running
ROOT --> sees
```

In [ ]: